Serial Decode — measured reference

Ian Ameline · version 1.56 · MIT licence

The detail behind MANUAL.md. Everything here was measured on a DMM6500 (firmware 1.7.17a) against an SDG2122X generator, verified where noted with an SDS1204X-E scope. The manual is written for someone using the app; this file is for someone who needs the numbers or is changing the code.

Every number in this file is a DMM6500 number. The app is expected to run on the wider Keithley TSP family — the README's **Which instruments** section sets out which models and on what grounds — but nothing here has been reproduced on any of them, and several of these figures are ones you would expect to move. The sample-rate ceiling, the press latencies, the display-object budget and the reading-buffer throughput are all properties of this instrument's hardware and firmware, not of the decoder. Treat them as measurements of one machine until someone repeats them.

The figures split into two classes, and only one of them should transfer. Anything descending from the **sample rate** — the rate ladder, the samples-per-bit table, the recording-length arithmetic — is timing, and a model at the same 1 MS/s inherits it unchanged. Anything descending from the **analog front end** is not: the **Tolerance envelope** below and **Levels and offsets** are measurements of one acquisition board.

The idle polarity on a line that never goes quiet belongs to neither class. It is decided by decoding both polarities and keeping the one the bytes support, with the two level voltages breaking a tie against `flatfloor` = 1.0 V — `lo < -1.0 and hi > +1.0` reads the line as RS-232, which rests at its negative level. That is arithmetic and a decode, with no acquisition board in it, so it transfers to every model unchanged.

The repeating-pattern rate misfit belongs to neither class. It is arbitration between two readings of the same pulse lengths, it reproduces with no acquisition board in the path at all, and it therefore transfers to every model unchanged — including the sample-rate dependence that drives it, since the candidates a reading admits depend on how many samples per bit the capture holds.

Those are the ones to re-measure first on another model, and on a DMM7510 they could plausibly move **in the app's favour**: its acquisition boards and digitizer are a significant step up, which is exactly the axis the threshold picker's histogram sensitivity lives on. A wider envelope would be welcome; a *shifted* one would quietly invalidate the table.

On a DAQ6510 there is no such split, and every figure here should hold. It shares both the UI board and the acquisition board with the DMM6500 — only the channel-board plugin differs — so the analog numbers are measurements of the same hardware, not of comparable hardware. That makes it the one model against which this file is a **falsifiable** document rather than an approximate one. It is therefore the first port to try and the cheapest to interpret.

Press latency

A front-panel **touch** press is **not dispatched while a script is running**, and presses **queue** rather than being dropped. So a handler's own duration *is* its press latency, and returning is the only way to answer a touch press. That is why a stop press is absorbed after the capture it interrupted.

The **front-panel TRIGGER key was meant to be the exception** — latched by the trigger hardware rather than delivered to the interpreter, so a running handler could see it. Measured 2026-08-18, it is not: the press does not reach the latch while a panel-initiated run executes, so **nothing makes a minutes-long press interruptible**. The run is bounded and says its cost beforehand.

Measured over 50 button presses covering every control in every state:

Press	Measured
Anything that does not capture	**77 ms or less** (Save is the slowest of them)
Capture, rate locked	**1.7 s**
Capture, unlocked, 9600 Bd and up	3.6 – 6.5 s
Capture, unlocked, 2400 Bd	4.9 – **9.1 s**
A recording, start to filed	**one press**, as long as the window takes — see below

The unlocked figure is a **range, not an average, and it is genuinely variable**: twelve back-to-back unlocked captures on one unchanged 2400 Bd line measured min 4.891 s, mean 5.358 s, max 7.253 s — a 2.36 s spread, 33 % of the maximum — and a separate run has been seen at 9.086 s. What varies is how many looks the rate probe needs, which depends on where the traffic is when the capture starts. `bench_panel.py` therefore bounds capture presses at **12 s**: margin sized to the measured spread rather than to a single sample.

An unlocked capture costs more the **slower** the line, which is the opposite of the intuition. The rate probe starts at 1 MS/s, where 20 000 samples is 20 ms — less than one frame below about 1200 Bd — so it steps its sample rate down until it catches traffic, and each step is another capture. Locking removes the probe entirely, which is why the first press auto-locks.

That a capture exceeds 500 ms is deliberate. A 240-byte window is 19 200 samples and reading one sample out of the capture buffer costs 21.7 µs, so the read alone is 0.42 s before any decoding happens. Splitting one capture across several presses would shorten the press, but one press for one capture is worth more.

A recording press is minutes, not milliseconds. One press records a window, decodes all of it and files it: at 9600 baud that is 34 s of recording plus ~110 s of decoding for 32 kB, or 8.5 s plus ~30 s for 8 kB. Decoding runs at ~300 byte/s end to end and does not scale with the baud rate. `bench_panel.py` bounds such a press at **300 s**, which is a hang detector rather than a latency budget — the duration is the window size the operator chose, and nothing shortens it.

Sample rates and samples per bit

FRAME mode and the streaming modes pick their sample rate differently:

- **FRAME** uses `fs_for_baud`, which oversamples from a discrete rate ladder, so the ratio is not constant: measured 8.33 samples/bit from 300 Bd to 19200 Bd, 8.68 at 115200 Bd, and 4.00 at 250000 Bd.
- **Streaming** uses `fs_for_burst`, the *lowest* listed rate that still delivers `minsabit` = 4 samples/bit, so it asks for ~4.17 samples/bit from 300 Bd to 19200 Bd and buys twice the recording length for it. Also ladder-dependent: 5.21 at 38400 Bd and 57600 Bd, 5.56 at 115200 Bd. The *delivered* ratio is slightly lower than the requested one, because `acq_overhead` costs ~0.51 us a sample.

Tolerance follows **samples per bit, not baud rate**, and the sample-rate ladder does not give every rate the same figure — 8.3 at 9600, 8.6 at 28800, 10.0 at 1500, 4.0 at 250 000. It holds at least ~8 up to 115200 Bd. Read `SA/BIT` rather than assuming a slow line is the safe one.

Recording length, and where the limits come from

There are **two recording modes**, capped at **8 192** and **32 768 bytes**. Nothing records without a byte ceiling, and beyond the ceiling the answer is flow control.

The two differ only in size, at every rate, and the choice is a responsiveness trade rather than a capability: one press records a window, decodes it and files it, so `8 kB` reaches bytes in about a quarter of the time, while `32 kB` spends less of the run on per-window overhead. Both are lossless.

The capture buffer holds **4 076 503 readings** (`ck_bufmax`) with the default buffers at their 100 000 defaults, out of the **7 080 255 readings** the instrument holds across all buffers together, measured. That figure is the app's own peak need — the largest capacity any baud, mode and frame format can ask for — so at the defaults nothing is ever shortened. The ceiling is derived at every launch from `defbuffer1.capacity` and `defbuffer2.capacity`, and launch first **frees the room it needs**: foreign buffers are deleted, then `defbuffer2` is resized to 100 000, then `defbuffer1`. A resize discards the buffer's readings, so each one is named in the USB log, and the capacities are restored at exit. Where a resize is refused the app shortens the recording rather than failing it. A mode asks for whatever its ceiling needs at the locked rate -- `cap x framebits x (fs / baud)` -- capped by the buffer. The table below is the 32 kB window; divide the readings and the durations by four for `8 kB`. Divide by `fs_for_burst` for the signal duration:

Locked rate	Readings	Signal	Wall clock to fill
300 Bd	2 730 667	1092 s	1092 s
1200 Bd	2 730 667	273 s	273 s
2400 Bd	2 730 667	136 s	136 s
4800 Bd	3 413 334	68 s	68 s
9600 Bd	3 413 334	34.1 s	34.1 s
19200 Bd	1 706 667	17.1 s	17.1 s
38400 Bd	1 706 667	8.5 s	17.1 s
57600 Bd	1 706 667	5.7 s	17.1 s
115200 Bd	1 820 445	2.8 s	18.2 s
153600 Bd	2 133 334	2.1 s	21.3 s

Signal and wall clock diverge above ~19200 Bd for the reason in the next section. Verified against the app's own arithmetic, not hand-computed.

The highest standard rate that records is 153600 Bd, and the cut-off is exact rather than approximate: `fs_for_burst` returns nil once `1/(minsabit * baud)` no longer exceeds `acq_overhead` plus one period of the top listed rate, which is `1/(4 * (1/1e6 + 5.1e-7))` = **165563 Bd**. 153600 clears it at 4.31 samples/bit delivered; 172800 is the next entry in the ladder and does not. Confirmed on the instrument 2026-08-18 on both sides of the boundary: an 8 kB recording at 153.6 kBd captures and decodes, and at 172.8 kBd the mode is refused. The refusal names the recording's own figure: "*192000 Bd: a recording gets 3.4 samples/bit, under the 4 floor -- use FRAME*" — deliberately not the `SA/BIT` header value, which is measured from a FRAME capture and reads about 5.2 at that rate. Recording needs 4 samples/bit and the digitizer stops at 1 MS/s, so 172800 Bd and beyond -- including the 250 kBd decode ceiling -- are FRAME only.

Nothing is dropped inside a recording. One hardware acquisition with no script running, so there are no gaps to fall through. Measured at 9600 Bd: **1925 of 1925 bytes contiguous** against a non-repeating payload, at a byte rate matching the wire to 0.5 %. Re-verified at 115200 Bd with the 1024-byte non-repeating `v71` payload: 400 checked bytes were **one unbroken slice** of it. And a full 2 800 000-reading recording at 4800 Bd decoded **67 749 bytes against 67 200 expected** for 140 s of line -- a 0.8 % match, with the ASCII gutter showing continuous text straight through the payload wrap.

Beyond one window: flow control, not a bigger buffer

A window is not a total. `Options > Rear BNC = FC Out` asserts `trigger.extout` once per armed capture -- `stream_arm` (serial_app.tsp), `acq_free` and `acq_triggered` (serial_core.tsp), with probe reads suppressed via `nofc` so a level-learning probe does not spend a credit. One pulse means "send up to one window". While Lua is decoding nothing is armed, so a device that waits for the credit cannot transmit into a closed window, and the byte log appends across windows. Unlimited total, bounded throughput.

And no interaction per window. `record_run()` loops credit -> record -> decode -> file -> credit inside the one press, so the operator presses Capture once for a transmission of any length. What ends it is under **The flow-control loop's bound** below.

Measured electrically, on the 50 Ω matched cabling, with the pulse read on the same split that feeds the generator: **idle-low at +0.20 V, high +4.45 V, peak +4.82 V, 5.72 µs wide** — and the width is the same at three thresholds (5.72 µs at 50 %, 5.71 at 2.5 V, 5.73 at 2.0 V), so it is a real width rather than an artefact of where it was measured. **Rise 10–90 % is 436 ns, 7.6 % of the pulse: the corners are square.**

The cabling is part of the measurement. On 1 m RG179 75 Ω, unterminated, the same pulse reads 4.92 V peak, a −0.96 V baseline and 9.4–9.8 µs — 1.7× the width, with an undershoot that is what a reflection on an unmatched path looks like. Re-measure the pulse on the path a receiver will actually see.

The width cannot be set on this firmware: `trigger.extout.pulsewidth` reads nil on 1.7.17a, confirmed again here, so `sdec.fc_pulse` (100 µs) is never applied. **A device waiting for a credit therefore needs an edge-triggered input, an interrupt or a latch** — 5.7 µs is under 1.5 bit times at 250 kBd, and a polling loop can miss it.

Still not closed as a loop, and now for a located reason. See *Triggering, in both directions* below: the generator's burst machinery does accept the arbs, but it did not act on this pulse.

The reading buffer accepts ~100 000 readings/s, and that is wall clock, not signal

The durations above are **signal** time — buffer size ÷ the sample rate the app asks for — and they are correct. What they are not is how long you wait. Measured fill rates, polling `buf.n` from the host while a press-driven recording ran:

Baud	Mode	Asked	Achieved	Ratio
4800	32 kB	20 kS/s	19 798/s	0.99×
9600	32 kB	40 kS/s	39 200/s	0.98×
19200	32 kB	80 kS/s	76 858/s	0.96×
38400	32 kB	200 kS/s	99 996/s	0.50×
57600	32 kB	300 kS/s	99 986/s	0.33×
115200	32 kB	640 kS/s	99 994/s	0.16×

Baud	Mode	Asked	Achieved	Ratio
160000	32 kB	1 MS/s	100 006/s	0.10×

This is not a configuration mistake and there is nothing to fix. It was measured three ways at `samplerate` = 1 MS/s, and all three land on the same ceiling:

Configuration	Readings	Time	Rate
`digitize.count = 1`, `SimpleLoop(200000)` — what the app does	200 000	2.017 s	99 146/s
`BLOCK_MEASURE_DIGITIZE` with count = 200 000	200 000	2.119 s	94 400/s
blocking `dmm.digitize.read(buf)`, `count` = 200 000	200 000	2.186 s	91 502/s
blocking `dmm.digitize.read(buf)`, `count` = 1 000 000	1 000 000	10.930 s	91 487/s

Two things from that table. **`SimpleLoop` takes exactly one reading per iteration and ignores `dmm.digitize.count`** — `count = 200000` with one iteration returns `n = 1`, so the loop count *is* the reading count on that path. And the blocking read, which is what FRAME mode uses and which genuinely samples at 1 MS/s, files readings at the same ~91 k/s. So the ceiling is the buffer write, not the digitizer and not the trigger model.

The digitizer really does sample at the rate it is told. At 115200 Bd the app asked for 640 kS/s and `acq_measure_fs` read back **479 677 S/s** with 4.16 samples/bit, and the bytes were contiguous and correct. The consequence is only wall clock: 505 723 readings gathered over 5.0 s of wall time represented **1.054 s of signal**, and the 12 171 bytes decoded match 1.054 s × 11 520 B/s to 0.2 %.

So above ~19200 Bd, filling the buffer takes roughly `capacity / 100 000` seconds regardless of rate — about 17-20 s for every mode in the table.

Stopping a run: there is no control that does it

A decode long enough to want stopping is a handler that has not returned, and a touch press is not delivered until it does — so the panel cannot offer a control, and the front-panel TRIGGER key does not deliver `trigger.EVENT_DISPLAY` while a panel-initiated run executes. A run therefore lasts as long as its size requires. That is why the size is chosen before the press and why the panel states the byte ceiling and shows a progress bar rather than an estimate.

The flow-control loop's bound

Under `FC Out`, one press credits the device, records a window, decodes it, files it and credits again, ending when a credited window comes back silent. A device that never stops talking would otherwise hold the panel indefinitely, so the loop has **two** backstops, and one is not enough:

bound	value	what it limits
`fc_maxwin`	32 windows	the BYTES — 1 MB at the 32 kB window size
`fc_maxsec`	1200 s (20 min)	the WALL CLOCK, decode included

A window count is not a time. Each window's acquisition is bounded independently at `min(stream_maxwait, strm_maxsec)`, and the decode costs more than the acquisition did — measured at 9600 baud, 34 s to fill a 32 kB window and about 109 s to decode it. So 32 windows is **76 minutes** there, and at 300 baud, where every window waits out the full 1200 s acquisition ceiling, the count alone allowed **ten and a half hours**. A panel that comes back the following afternoon is a power cycle.

So the clock is the bound that fires first: 20 minutes is roughly **eight** windows at 9600 baud, not 32. That is a deliberate reduction in bytes per press, and it costs no data — the sender only transmits when credited, so it is still waiting where the run stopped. What it costs is two presses and a file boundary, described below.

Both are backstops, and they are the ONLY things that end a run early — no key does. When either fires the note row names which one and gives the remedy, **from the first window onward** — a run stopped by the clock on window one is the normal case on a slow line.

The remedy is `Mode`, then `Capture`, and it continues in a NEW file. Not one press, and not the same file: every way out of a recording goes through `mode_exit()`, which sets `capmode` back to `frame` *and* nils `flog_path`. So `Capture` on its own takes a frame capture, and the bytes that follow land in the next numbered `bytesNNN.txt`. **The data is not lost** — the device is still waiting for a credit that has not gone out, so nothing was transmitted in the gap — but a bounded run is *fragmented across two files*, not truncated.

> **Open decision for a future version.** If a bounded run should continue into the same file, that means keeping `flog_path` across a `mode_exit()` that ends on a bound — which interacts with `NewLog`'s "closed *file* — the next capture starts a new file" note and with the per-capture eviction rules. That is a design change rather than a fix, so it is recorded here and not made.

The idle watchdog and remembered levels

A recording's quiet-line exit needs a voltage threshold, and `clear_result()` zeroes the measured one at the top of every capture — so `stream_begin()` probes the line first. **Under flow control that probe looks at a silent line**, because the device is waiting for its credit, and the same is true of every window after the first. Without a fallback the watchdog would be disarmed exactly where it matters most, and every window would wait out `strm_maxsec`.

So `sig_levels()` keeps the last levels it successfully measured (`lvl_thr`, `lvl_hyst`, `lvl_swing`), past `clear_result()`, and `stream_begin()` falls back to them when its own probe finds nothing. They exist whenever a rate is locked, and locking a rate is already required to enter a recording mode. With no remembered levels the watchdog stays disarmed, which is the safe answer: a truncated recording is worse than a long one.

`display.waitevent(0)` blocks and wedges the instrument. Do not use it. The reference documents it as `<object id>, <sub id> = display.waitevent([timeout])`, and the instrument's own display example polls it with a 0 timeout inside a long loop to catch a Stop press.

On firmware 1.7.17a a 0 timeout does not mean "return immediately with whatever is pending". With no waitevent-enabled object to report, it waits: the TSP interpreter stops answering, the control socket goes silent, and reconnecting does not recover it. Only a power cycle does. Confirmed both with the app running and on a bare instrument with nothing built. Recorded as `DMM_WAITEVENT_WEDGES_THE_INSTRUMENT` in `tools/instruments.py`.

A second obstacle sits behind it: an object's event slot is *either* a `waitevent` flag *or* a command string, never both (Display API reference, `setevent`), and every button in this app is a command string.

`display.OBJ_TIMER` is **not** a DMM6500 object type. It appears nowhere in this instrument's display API reference, so there is no timer object to drive work from.

Tolerance envelope

Measured at 8 samples per bit. In every case beyond the limit the failure is **reported** rather than silent, except where noted.

Condition	Byte-exact to
Clock error, rate **locked**	**±4 %** — past this, running *fast* corrupts bytes silently
Clock error, **auto-detect**	±12 % and beyond — it measures rather than assumes
Timing jitter	**±15 %** of a bit time — ±15.6 µs at 9600 Bd, ±1.3 µs at 115200. Past ±20 % it can fail silently
Amplitude noise	~40 % of the logic swing — ~1.3 V p-p on a 3.3 V line, which the panel reports as **13 dB**
Low-pass filtering (poor probe, long cable)	RC up to **0.7 bit times** — a ~2.2 kHz filter at 9600 Bd
Slow edges (open-drain pull-up)	rise up to ~40 % of a bit time — 42 µs at 9600 Bd, 3.5 µs at 115200
Impulse spikes / dropouts	usually costs bytes and raises errors; a spike confined to a *data* bit changes that byte undetectably
Logic swing	**0.33 V to 8 V** two-level, and up to **9.3 V p-p** where the extra span is spikes — the limit is the swing, not the family
DC offset	anything keeping both levels inside ±10 V

The UART framing cliff itself is 0.5/9.5 = 5.26 %; `ratemargin` = 0.04 is where the app starts warning.

Levels and offsets, verified

Byte-exact from –5 V to +5 V of offset with a 3.3 V swing, and across the same offset range with a **1.6 V** swing — including 1.6 V sitting at 5.05…6.65 V and at –5.06…–3.46 V. A 60 mV swing is refused -- by the swing floor (`minswing` = 0.10 V, which reports the measured swing) or as a line with no transitions, depending on which test trips first. The useful floor is between the two.

The floor is 0.12 V, so 0.33 V carries nearly three times the margin it needs. Swept through the app's own `sig_levels`, `sig_edges`, `sig_idle` and `decode_from` at the rates it picks itself — 8 sub-bit phases × 9600 and 38400 Bd × four noise levels, a 236-byte payload riding a 2 V offset — every swing from **0.12 V to 9.30 V** decodes byte-exact at every noise level, 16 points of 16 each. 0.10 V refuses: it sits exactly on `minswing` and the trimmed histogram measures 0.0996 V, just under. A 60 mV swing refuses at every noise level, and the swing it *reports* grows with the noise, 0.06 V to 0.16 V at 50 mV peak, because peak-to-peak rises while the logic swing does not.

The 0.5 V end of the range is confirmed on the instrument, not only offline. The `levels` stage of `tools/bench_smoke.py` plays one waveform at **5 V, 3.3 V, 1.6 V, 1.0 V, 0.5 V, 0.33 V and 0.25 V** of swing and takes one in-app Capture at each, checking the `LOGIC` cell against the swing the app measured, that `THRESH` landed within a quarter of the swing of mid-swing, and the bytes. 0.25 V is below the claimed floor deliberately: a floor with nothing measured beneath it is a guess.

The noise budget is a fraction of the swing, not a voltage. The generator's `noise` is a peak amplitude in volts, so the same 50 mV is 1.5 % of a 3.3 V swing and 10 % of a 0.5 V one. A small swing does not fail because it is small; it fails when the noise on it reaches the ~40 % above.

Signal-to-noise, and the cliff it warns about

The `S/N` cell reports the noise sitting on the two logic levels, in dB, as $20 \cdot \log_{10}(\text{swing} / \text{noise}_{\text{rms}})$. It is unsigned because it is a ratio of signal over noise, and positive by construction: a line whose noise exceeds its swing has no logic levels, and `sig_levels` has already refused it before the figure is computed.

Edges are excluded by position, not by voltage. A sample partway up an edge is indistinguishable from a noisy one by amplitude, so an amplitude window measures **slew** and calls it noise. Measured on a **noiseless** capture, a `thr ± hyst` window reports 24.8 dB and a within-15 %-of-level window 38.4 dB — both of them pure edge slew — and **neither figure moves** when real noise is injected 40 dB below. Requiring both immediate neighbours on the same side of `thr ± hyst` drops every sample within one place of a transition; that tracks injected noise to 0.3 dB from 24.8 dB down to 84.8 dB, and costs 23 % of the samples.

~1000 samples, strided across the whole capture rather than taken from the front. A thousand contiguous samples at 1 MS/s span 1 ms — a twentieth of a 50 Hz cycle — so mains pickup inside that window is a slow tilt that reads as signal rather than as noise; and at the head of a capture they can all land in the opening idle, where only one of the two levels exists. Residuals are taken about each level's own mean and pooled, because measuring about the threshold would report half the swing as noise.

It is not derived from `wlo` / `whi`. Those are histogram-quantised — `nb = floor(n/binocc)` capped at 512 bins, ~23 mV per bin over 3.3 V — which would cap the reportable figure near 43 dB at a 3.3 V swing and 21 dB at 0.25 V, both well inside the range that matters.

What it reads on the bench

Measured through the app's own Capture on a DMM6500, one waveform at 9600 Bd 8N1 played at seven swings, **and on two kinds of wiring**. The right-hand column is deliberately bad: a **6 ft unshielded silicone test lead wrapped three times around the instrument's own mains cord** for signal, against a **3 ft** ground lying on top of the instrument — unshielded, length-mismatched, and coupled to the mains on purpose, because matched shielded RG316 of equal lengths is not what anyone debugging a bit-banged UART actually has.

swing delivered	matched RG316	unshielded, mains-coupled	cost
5.007 V	80 dB	76 dB	–3.4
3.302 V	80 dB	75 dB	–4.7
1.599 V	76 dB	73 dB	–3.2
1.000 V	74 dB	72 dB	–2.0
0.500 V	68 dB	66 dB	–2.6
0.330 V	65 dB	63 dB	–1.7
0.250 V	64 dB	60 dB	–3.2

Every point decoded byte-exact on both, 7 of 7 either way, and the swing the app measured was identical to the millivolt across the two — 5.007, 3.302, 1.599, 1.000, 0.500, 0.330, 0.250 V — so the wiring moved the noise and nothing else.

Deliberately awful wiring costs 2 to 5 dB. It is nowhere near the 15 dB amber threshold, let alone the 12 dB cliff, so on this bench the figure is dominated by the instrument's own front end rather than by the leads. Piling active electronics onto the same lead — a powered-on iPhone 14 Pro and two running HP calculators

resting on it — moved nothing: 7 of 7 again, and 59.6 to 79.8 dB against the 60.3 to 76.4 dB without them, which is inside the run-to-run spread. The measured noise is **0.2 mV to 0.5 mV rms** on good cable and 0.2 mV to 0.8 mV on bad, i.e. roughly constant in **volts** — which is why the reported dB tracks the swing so closely, falling about 6 dB for every halving of it.

A larger swing is not automatically quieter. 3.302 V reads the same 80 dB as 5.007 V and sometimes better, because the generator's own output noise scales with its amplitude while the instrument's floor does not. The two effects cross somewhere around 3 V on this bench.

Below that crossover the figure is reporting the meter, and the ~6 dB per halving of swing is the evidence. Inference rather than measurement, but it follows from where the parts sit: the generator's amplitude control is a relay-switched resistor ladder *after* its DAC, so it attenuates that DAC's noise along with the signal — a generator-dominated figure would therefore stay roughly flat as the amplitude fell. Falling 6 dB per octave instead means the dominant noise is constant in **volts** and downstream of the attenuator, which is the instrument's own front end. That is also why deliberately bad cabling costs only a few dB: on this bench the leads are never the limit.

The figure is not comparable between FRAME mode and a recording, and the reason is not yet known. A 32 kB recording of the same 9600 Bd 3.3 V line reported **35 dB** where FRAME mode reads 80 dB. Both decoded byte-exact, so this is a reporting discrepancy rather than a decode one — but until it is explained, read `S/N` against other captures in the same mode and not across modes.

Two causes are ruled out by measurement:

- **Not the oversampling.** Swept from 4 to 32 samples per bit on a noiseless capture, the reported figure is the 1 µV floor — **exactly zero measurement error** — at every oversampling, and it stays there with the digitizer's aperture modelled from instantaneous to fully integrating. `sig_noise`'s run gate excludes every partial sample at 4 samples per bit as completely as at 32.
- **Not the window length against mains.** With 10 mV of 60 Hz hum injected, a 25 ms window — 1.5 cycles — reports 53.4 dB and an 8.5 s window reports 53.6: the same answer, and the correct one for that hum's rms. A window shorter than a mains cycle sees hum as well as a long one does.

The next candidate is the `lvl_*` threshold cache, which is a known history-dependence in the recording path specifically: a threshold or hysteresis carried over from a different capture would misplace the band the run gate uses.

The floor at 0.1 V is the app's own, not the noise

Measured on the bench at **19 200 Bd 8N1, 0.1 V p-p**, threshold 0.06 V: it decodes, and reports **58 dB**. That implies a noise floor of **0.13 mV rms** — quieter still than the 0.2 mV at larger swings, because the generator contributes less noise at a smaller amplitude — and it leaves **46 dB of margin above the 12 dB cliff**. Nothing about the signal is marginal at 0.1 V.

What refuses below it is `sdec.minswing` = **0.10 V**, a declared guard: `hi − lo < minswing` means the line is dead, and a dead line is refused rather than decoded. So 0.1 V p-p sits *exactly on* that boundary and which side it falls is decided by the trimmed histogram, not by the noise — the same 0.1 V measured 0.0996 V offline and refused, and measured a shade over on the bench and decoded.

The bracket is two generator settings wide. At **400 mV** of generator amplitude the line is 0.1 V p-p and decodes; at **300 mV** it is 0.075 V and does not. Those two straddle the 0.10 V guard, and nothing else changed between them — so the transition from decoding to refusing happens at the declared floor rather than anywhere the noise could put it.

That is why the published floor stays at 0.33 V: it is the smallest swing with margin against the *guard*, not against the noise, and the noise budget has three orders of magnitude in hand. Lowering the claim below 0.33 V is a change to `minswing`, not to this document — and whatever became the new limit would be the histogram's bin resolution or the threshold picker, since at 0.13 mV rms the 12 dB cliff does not arrive until about 0.5 mV of swing.

Where the decode actually fails

Swept offline at 9600 Bd, 12 capture start phases per point, injecting uniform noise as a fraction of the logic swing and running the app's own `sig_levels` → `sig_edges` → `sig_idle` → `decode_from`:

noise, peak	reported `S/N`	byte-exact	at the swing
0 %	>99 dB (clamped)	12 of 12	3.3 V, 1.0 V, 0.5 V
8 %	26.7 dB	12 of 12	all three
20 %	18.7 dB	12 of 12	all three
35 %	13.8 dB	12 of 12	all three
45 %	12.5 dB	12 of 12	all three
50 %	—	**0 of 12 — all twelve refused**	all three
60 %	10.3 dB	0 of 12, 8–9 refused	all three

The cliff is at 12 dB, and past it the app refuses rather than guessing. There is no band where it decodes confidently and wrongly: between 45 % and 50 % of the swing it goes from byte-exact on every capture to `no clear logic levels` on every capture.

The same dB figures at 3.3 V, 1.0 V and 0.5 V of swing, agreeing to 0.1 dB. That is the measurement behind "the budget is a fraction of the swing": the voltage at which a line fails scales with the swing, and the *dB* at which it fails does not move at all. It is also what justifies one pair of thresholds on the panel for every line the app will ever meet.

The sweep doubles as an end-to-end check of the measurement itself: predicted $20 \cdot \log_{10}(\sqrt{3}/\text{frac})$ against reported, over 14 noise levels and three swings, agrees within **0.5 dB** everywhere except where the prediction is meaningless (0 % noise) or the levels are collapsing (60 %).

The bands

	green	amber	red
`FIT`	≥ 0.90	0.72 … 0.89	< 0.72
`S/N`	≥ 30 dB	15 … 29 dB	< 15 dB

`FIT` 0.90 is `autolock_fitq` itself, so green means "good enough for the app to pin the rate". 0.72 sits 10 % above a structural knee at 0.65: `sig_fit`'s quality is (1 − mean_err/T) × (nfit/nsmall) and only widths within 0.35·T of a multiple count, so tightness alone cannot take it below 0.65 and anything under that means *coverage* is collapsing rather than tightness.

`S/N` green at **30 dB** is noise under ~5.5 % of the swing; **red at 15 dB** is noise around a third of it, which still decodes 12 of 12 in the sweep above. **Both thresholds sit above the 12 dB cliff on purpose** — a band that turned red only at the cliff would light up on captures that were about to refuse anyway, which

warns nobody. The 8 dB figure that `20·log10(1/0.4)` gives from the 40 % noise tolerance is not the cliff in these units and nothing should be derived that way: that tolerance is a **peak** amplitude and this cell is an **RMS** one, and the rms equivalent of 40 % peak is 12.7 dB.

Both bands are keyed on the number the cell **prints**, rounded exactly as the cell rounds it, so the colour can never contradict the figure beside it. Banding the raw value instead is visibly wrong on the panel: a raw 0.8996 prints as `0.90` and would be coloured as the 0.899 it really is, putting an amber figure on a green threshold. A nil figure prints `--` and keeps the column's own colour, because a red dash reports a bad measurement where there is no measurement.

The swing is a swept axis. Every plan cell draws its own target swing — uniformly from **0.45 V to 8 V** — and then a DC offset from whatever is left inside ±9.5 V. One 1677-cell hardware lap drove **1.652 V to 9.300 V p-p** and 1634 cells decoded byte-exact. The largest **two-level** swing that decoded is **8.000 V**, on the LIN vectors at 2400, 9600, 38400 and 76800 Bd; the largest span of any kind is **9.300 V p-p**, on the impulse-spike vector, whose ±3 V transients ride a 3.3 V logic swing.

And the failure rate barely moves with the swing. 100 offline laps of the plan the instrument itself runs — **133 300 cells and 1 066 400 decodes**, eight capture phases a cell — with each cell's swing drawn independently and recovered from the plan it was drawn into:

drawn swing	cells	decodes	failed	rate / bytes	rate
0.45 … 1.0 V	9 733	77 864	528	501 / 27	0.678 %
1.0 … 2.0 V	17 616	140 928	954	915 / 38	0.677 %
2.0 … 3.5 V	26 237	209 896	1 354	1 299 / 55	0.645 %
3.5 … 5.0 V	26 730	213 840	1 252	1 203 / 49	0.585 %
5.0 … 6.5 V	26 365	210 920	1 496	1 438 / 58	0.709 %
6.5 … 8.0 V	26 619	212 952	1 295	1 241 / 53	0.608 %

Within a factor of 1.21 across the whole range, and not monotonic — the best band is 3.5–5.0 V, the worst 5.0–6.5 V, and the narrowest swing is neither. The `rate / bytes` column is the split defined below and it holds nearly the same proportion in every band, so **neither failure has a useful trend against the swing**.

That is not the same as independence, and the run is large enough to tell the difference. Across the six bands the counts differ by more than chance — Pearson $\chi^2 = 33.5$ on 5 degrees of freedom — so something band-dependent is present at the 0.1 percentage-point scale. What the measurement supports is that the effect is small, unordered and not a design limit; what it does not support is "the swing does not matter". Any apparent dependence should be checked against the vertical draw in BENCH.md first, since a band placed across ground is read as RS-232 and fails for a reason that has nothing to do with its size.

Rate detection and decode, counted separately

These are two unrelated failures and one blended number hides which of them moved. A capture can name the wrong baud rate, or it can name the right one and still hand back the wrong bytes. The same 100 laps, **1 066 400 decodes**, against a plan built to attack the decoder:

	decodes	of all decodes	of the failures
the reported rate is wrong	6 597	0.619 %	**95.9 %**
— and the bytes are right anyway	2 621	0.246 %	38.1 %

	decodes	of all decodes	of the failures
— and the bytes are wrong too	3 976	0.373 %	57.8 %
the rate is right, the bytes are wrong	280	**0.026 %**	4.1 %
nothing decoded where something should be	2	0.0002 %	0.03 %
more frames flagged than the app's own budget allows	**0**	—	—
total	6 879	0.645 %	100 %

Given the right rate, a byte comes out wrong on 0.026 % of decodes — one in 3 809 — against 0.619 % where the number on the panel is wrong. Rate detection is 24 times more likely to be the fault than framing and sampling are, and **38 % of the rate failures decode every byte correctly** and only mislabel the rate: that is the cluster-C class under **Ambiguity** below, and on the panel it is a wrong `BAUD` cell above a correct dump. Of 133 300 cells, 1 415 (1.06 %) failed at **at least one** of their eight capture phases and **617 (0.46 %) failed at all eight**. Only the second figure is a property of the cell rather than of where the capture happened to open, and it is the one to set beside a bench lap, which asks once.

By standard rate. Each row is 31 waveforms a lap over 100 laps — **3 100 cells, 24 800 decodes**:

standard rate	rate wrong	bytes wrong	nothing decoded	per 10 000 decodes
300	9	0	0	3.6
600	11	0	0	4.4
1200	18	0	0	7.3
1800	12	0	0	4.8
2400	5	0	0	2.0
4800	8	0	0	3.2
7200	5	0	0	2.0
9600	12	0	0	4.8
14400	14	0	0	5.6
19200	14	0	0	5.6
28800	0	7	0	2.8
31250	0	19	0	7.7
38400	**0**	**0**	0	**0**
57600	0	12	0	4.8
76800	**0**	**0**	0	**0**
115200	0	8	0	3.2
128000	0	1	0	0.4
153600	0	0	2	0.8
172800	**0**	**0**	0	**0**
192000	**0**	**0**	0	**0**
230400	**0**	**0**	0	**0**

standard rate	rate wrong	bytes wrong	nothing decoded	per 10 000 decodes
250000	0	44	0	17.7
all 22	**108**	**91**	**2**	**3.7**

201 failures in 545 600 decodes at standard rates — 0.037 % — against **6 678 in 520 800 at the drawn non-standard ones, 1.28 %**. **A device sitting on the ladder is 35 times better off than one between its rungs**, which is where the interstitial rates in the plan exist to put pressure.

And the split inverts at 28 800 Bd. Every standard-rate failure from 300 to 19 200 is a misreported rate with **not one wrong byte**; from 28 800 up, every one is a wrong byte with **the rate correct**. Five standard rates — 38 400, 76 800, 172 800, 192 000 and 230 400 — produced no failure of either kind in 24 800 decodes each. The worst row is 250 000 Bd, which the 1 MS/s ceiling digitises at 4.00 samples a bit, the app's own floor; the two decodes that produced nothing at all are both at 153 600 Bd. See **Sample rates and samples per bit** above for what each rate is sampled at.

On hardware, a 1 kB non-repeating payload is byte-exact at all eleven of 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200 and 250000 Bd, with no flagged bytes at any of them.

8 V is a generator ceiling, not a decoder one. The LIN vectors render 0…6 V inside a 15 Vpp full scale, so 1.6× reaches 24 Vpp and clamps to the SDG's 20 Vpp — 6.0 × 20/15 = exactly 8.000 V. No clean two-level vector can be driven past it by amplitude alone, and nothing in the decoder objects at 8 V.

Failure rate does not track the swing. Per band over that lap, in V p-p: 1.6–2.5 1.9 % of 324 cells, 2.5–4.0 2.3 % of 659, 4.0–5.0 2.3 % of 427, 5.0–6.5 5.1 % of 156, 6.5–8.0 3.8 % of 79, 8.0–9.3 3.1 % of 32. The peak is mid-range rather than at either end, and the 5.0–6.5 band's eight failures are the LIN and periodic-payload vectors, which sit at those swings by construction and fail on **rate**, not on level. Reading a swing effect off these bands needs the vector held constant: only six of the 41 can be driven past 6.6 V p-p at all, and every one of them is an impairment waveform.

Rates verified on hardware

43 matrix points and 19 non-standard rates, byte-exact, with no instrument event logged: six frame formats, the standard ladder 300 – 250 000 Bd, a 1 kB non-repeating payload, three named logic swings and twelve DC offsets. The plan stage sweeps the swing continuously on top of that ladder — 1.652 V to 9.300 V p-p across one 1677-cell lap.

Non-standard rates matter because a device with an awkward crystal divisor does not sit on the standard ladder: 900, 1500, 2800, 3600, 7200, 12000 and 16000 Bd all decode byte-exact, as do arbitrary values like 1379, 2731, 5333, 8123, 13333, 21600, 29127, 41666, 53333, 71111, 89123 and 104857.

A rate within **2 %** (`snaptol`) of a standard one is *reported* as the standard one — 29127 Bd reads as 28800 — because real devices use standard rates and 2 % is far inside the framing margin. The bytes are unaffected.

Above the 250 kBd ceiling it refuses by name rather than reporting a submultiple: 255 kBd, 260 kBd, 300 kBd, 400 kBd and 921600 Bd were each declined by the baud they imply — "255026 baud is past the 250 kBd ceiling" — rather than by a samples-per-bit count. The two are the same limit only at 1 MSa/s, where the 4 samples/bit floor is exactly 250 kBd; below that sample rate a bit time under 4 samples is an ordinary rate captured too slowly, so it is refused by naming the capture rate instead.

The panel's marginal-rate warning follows the same rule, and it matters more there because that warning sits beside a decode that *succeeded*. At 1 MS/s "only 4.0 samples/bit — at or past the 250 kBd ceiling" is true; at 20 kS/s the same 4.0 samples/bit is a 5000 baud line, a fiftieth of the ceiling, and the warning names the capture rate it has run out of instead.

100 baud is the declared floor (`sdec.minbaud`), refused by name the way the 250 kBd ceiling is, and it carries `plaustol` at both ends so a rate sitting on a limit is not rejected for a rounding error. Nothing below it is supported: `stdbaud` starts at 110 Bd, and a rate typed under 110 reads as 0, meaning auto-detect.

That floor is what rules out relay-driven links. A mechanical relay must be settled before the receiver's mid-bit sample, so switching time caps the rate at `1 / (2 · t_settle)` — about 100 Bd for a 5 ms micro-relay, putting 300 Bd out of reach threefold and 110 Bd short by a tenth. The rates that would actually fit are 75 Bd and below, which is under the floor. Relays make a poor transmitter regardless: make and break times differ, so each edge carries a systematic offset; contact bounce adds edges inside the start bit, which surfaces as rejected false starts rather than framing errors; and at ~55 operations per second a 110 Bd link spends a 10^5-cycle contact rating in half an hour. The sample-rate ladder is not the obstacle — 1 kS/s is 9 samples/bit at 110 Bd — and no figure in this document is measured below 300 Bd.

Ambiguity, and what FIT does not tell you

`FIT` is not a probability that the rate is right. It measures how well one bit period explains the measured pulse widths — and on its own it cannot separate a rate from **double** that rate. Every pulse that is a whole number of bit times is also a whole number of *half* bit times, so both readings fit equally well: measured 0.99999999 for each, on the same capture. The halved reading then finds twice as many frames, which any error count rewards.

So a shorter bit time is rejected on separate evidence, by two tests that divide the work between them. Both are consulted only when the measurement itself lands on a standard rate; where it does not, a rescaling is the only route to the truth and neither test runs.

- **Some pulse must span an odd number of the candidate's bit cells.** Real traffic has single- and three-bit runs, and at half the true rate every pulse spans an even count. Only runs up to 12 bit times count, because a longer one is inter-byte idle whose length is set by when the capture started rather than by the format. Measured over 368 640 decodes spanning every capture start position: at a halved bit time no start position misreads the rate.
- **The short end of the widths must not span more than ~2.5 of the candidate's bit cells.** The test above is arithmetically confined to even divisors — halving, quartering and sixthing a bit time make every count even, while dividing by an odd number leaves the counts' parity exactly as it was, so thirds and fifths pass it whatever the payload. What separates those is the minimum: ordinary traffic has single-bit runs, since a ten-bit frame opens with a one-bit start bit. Measured on one capture, the short end against the candidate's own bit time: **1.00** at the truth, 2.00 at a half, 3.00 at a third, 5.00 at a fifth. **The bound must sit below 3**, because a 3x misfit lands at exactly 3.00 and that is the largest family — measured on the LIN-break vector at 19 200, 57 600 and 76 800 Bd over 60 capture openings each, 2.0, 2.5 and 2.9 report the true rate every time while 3.0 and 3.5 give back 42, 60 and 60 wrong answers. 2.5 is the middle of that range, so the bound sits on neither edge. **What it costs:** a payload whose shortest genuine run is three bit times or more — `0x00` with a two-bit gap — cannot have a third-of-the-rate reading admitted once the fit snaps; measured, that capture reports a third of its true rate with the test and without it alike. The **5th percentile** of the widths is used rather than the outright

minimum, since one narrow glitch would otherwise silence it — and such a glitch is itself a route to a fifth-of-the-bit-time reading.

Samples per bit decides how many candidates are admissible at all, which is why the rate a capture is digitised at matters as much as the waveform. A reading is refused outright below 4 samples per bit, so at the ~8 samples/bit a locked rate gives, only the halved candidate clears that floor; at the 1 MS/s the first probe of an unlocked capture uses, a 31 250 Bd line arrives at 32 samples per bit and every divisor from 2 to 6 clears it. The probe pass is therefore where a misfit is decided, and it keeps its own answer whenever no lower listed rate would oversample adequately.

Neither test touches the *longer*-bit-time direction, so the genuine ambiguity below is preserved, and the note row still names a rival rate when one really fits.

Perfectly periodic traffic can be genuinely undecidable: eight `0x00` frames at 9600 Bd 7N1 with a one-bit gap are bit-for-bit the same waveform as four `0x08` bytes at 4800 Bd 8N1. No decoder can tell which device sent it, so the app says so instead of guessing. Irregular gaps between bytes remove the ambiguity entirely.

A capture beginning mid-byte costs **2 to 12 bytes**, measured across thirty baud rates, and does not scale with the rate — it is at most one frame plus the trigger's own offset. Those bytes are displayed but their bit boundaries are wrong, so they are arbitrary groupings of real wire bits.

Data Bits — why `Auto (any)` is not the default

`Auto (7/8)` searches 7 and 8 data bits. `Auto (any)` also tries 5, 6 and 9. Measured over 105 hostile-signal cases, the wider search changed the answer in **5** of them, and only **1** was a genuine 9-bit stream: the other 4 were damaged captures that a wider frame made look *cleaner* while being wrong, because the extra data cell absorbs whatever broke the stop bit. 9 data bits is searched **only** when explicitly forced, for the same reason.

Only one stop bit is searched and reported: a second stop bit is indistinguishable from a bit time of idle, which the decoder already tolerates. A 2-stop line decodes correctly and reports 1 stop.

Auto-lock conditions

Auto-lock is on by default and conditional. It locks only when **all** of these hold:

- the rate is above the 100 Bd floor and below the 250 kBd ceiling
- at least 8 good frames, and a clean unbroken run covering ~60 % of the capture
- `FIT` at 0.9 or better
- the logic levels were stable — no drifting or noisy baseline

A non-standard rate locks too, deliberately. The first condition asks only whether the figure is a rate at all; landing on the standard ladder is not required, so a device on an awkward divisor is lockable and its recording modes work without a hand-typed number. The three conditions after it are what judge the evidence — snapping would judge only tidiness.

What gets locked is rounded, not the raw measurement: the coarsest step that moves the rate under 1 % — nearest 100 Bd above 10 kBd, else nearest 10, else whole baud. So a device set to 16100 Bd measuring 16099 locks as **16100**, while 105 Bd locks as 105 rather than being dragged to 110. A rate

already on the standard ladder is never moved, which is what keeps MIDI's 31250 from becoming 31300.

Otherwise the padlock stays amber and **Lock Rate** is offered. Polarity is **never** locked; it is re-derived every capture, because freezing it off one short detect capture is how a confidently inverted, entirely wrong decode happens.

Flow control

`Options ▸ Rear BNC = FC Out` makes the rear EXT TRIG OUT emit **one** pulse once the digitizer is armed: idle-low at **+0.20 V, +4.45 V high, 5.72 µs wide, 436 ns rise** on a scope — the same figures given above, not the superseded 75 Ω-cabling ones. The width is the firmware's own and cannot be set here, so the receiving device needs an edge-triggered input rather than a polling loop. It is a **credit, not CTS**: treat the pulse as permission to send no more than one capture can hold. It does nothing unless the device is built to wait for it.

One press then runs the whole conversation, one credit per window, until a credited window comes back silent or the 32-window bound fires. The device has to go quiet when it has nothing to send: that silence is the only signal the app has that the transmission is over.

Triggering, in both directions: what is tested

The bench wires both directions permanently — SDG CH2 into the DMM's rear TRIGGER IN, and the DMM's rear TRIGGER OUT into the generator's rear In/Out — with every line also on a scope channel. What follows separates what has been **measured** from what has not, in each direction.

Into the DMM: an external pulse starts a decode — WORKS

the marker, measured on the scope	**0 to +4.972 V**, 50.00 µs wide, **100 ns** rise, +8 mV overshoot
against the DMM's EXT TRIG IN spec	≥ 1 µs wide, 0–5 V TTL — **50× the width floor, 3 V of level margin**
pulse present, quiet line	capture completes in **1.1 s**
pulse absent, quiet line	**refuses after the full wait**, `edge trigger unavailable; captured free-running`

The negative is the evidence. `acq_triggered()` falls back to a free-running capture when no trigger arrives, and a free-run on a quiet line still returns samples — so "it completed" proves nothing on its own. What proves it is that removing the pulse changes the outcome *and* names the reason.

`Rear BNC` alone is not enough for an anchored capture, and `sdec.trigext_only` is why. `trigext` **OR**s the rear input with the analog start-bit trigger — the right default, because "start on a start bit, or when the DUT asserts its GPIO, whichever comes first" is not expressible as a single choice. But on a line that never stops transmitting the start bit always wins, so the external pulse cannot decide *where* the window opens. Measured on a live line:

mode	marker	result
`trigext` (OR)	off	completes in **1.1 s**, `trigblended = true` — the start bit won

mode	marker	result
`trigext_only`	off	**times out at 6 s**, `trigblended = false` — the busy line is ignored
`trigext_only`	on	completes in **1.2 s** — the marker armed it

The middle row is the one that cannot be faked: without exclusivity a busy line completes every capture. `sdec.trigext_only` is **off by default** and has no panel control — it makes a capture depend on equipment the product does not require, and a user with one probe must never wait on a pulse that cannot arrive.

So `Rear BNC` means something different in each of the three Trigger modes, and only two of them are useful:

`Options ▸ Trigger`	what the rear input does	how
`Start bit`	**OR'd** with the analog start-bit trigger	`blender[1]` over `EVENT_ANALOGTRIGGER` + `EVENT_EXTERNAL`, `orenable = true`
`Trigger key`	**OR'd** with the front key — the only competing source is the operator, so this is the panel-reachable way to make the pulse matter on a busy line	same blender, `EVENT_DISPLAY` + `EVENT_EXTERNAL`
`Free run`	**ignored entirely**	the blender branch is guarded `sdec.trigmode ~= 'free'`, and free run never reaches `acq_triggered()` at all

The Free-run case is reported, and by the one mechanism that cannot be reached by the wrong path. `sdec.options_apply()` assigns `sdec.trigext` without a note of its own, and the one-shot lognote in `sdec.trigext_toggle()` fires only if that speculatively-wired status-row rect ever delivers a press — so neither is what carries the warning. `sdec.ui_notes()` **derives it from state instead of announcing it on change**: while the combination holds, every refresh re-adds `Rear BNC = Trig In is set but IGNORED -- Free run means do not wait for anything`, and `ui_trig_t` appends a `?` to `TRIG IN`. A recomputed note cannot be missed by arriving through a path nobody thought to instrument, which is why the setting-looks-applied-but-is-not class of warning belongs there and not in a handler.

When the blender is unavailable the code falls back to the rear BNC **alone** and says so (`no trigger blender; using the rear BNC alone`) rather than dropping the operator's request. Four-case regression proof: `tools/bench_trigin.py`, to be re-run after any change to `acq_triggered()`.

What is NOT yet shown is anchoring to a payload offset. That needs the marker phase-locked to the signal arb, which needs a marker waveform uploaded alongside it. Proven here is that an external pulse, and only that pulse, starts the capture.

Out of the DMM: a credit makes the generator send more — NOT YET

Three gates, in order, and it fails at the third:

gate	result
shape of the credit pulse	**PASS** — idle-low, +4.45 V, 5.72 µs, 436 ns rise, square corners (above)

gate	result
does burst keep TrueArb?	**PASS** — `BTWV STATE,ON` leaves `SRATE MODE,TARB,VALUE,96000Sa/s` intact, and `STPS,0` needs no change. This was the make-or-break: had burst forced DDS, every stored vector would be resampled and interpolated, and none of them would be valid stimulus any more
does the generator act on the pulse?	**FAIL** — with `TRSR,EXT`, `GATE_NCYC,NCYC`, `TIME,1` and the output on, the generator stayed silent through a credit

The negative control holds: armed and uncredited, the generator was silent for 2.5 s and the scope stayed `Ready`, so "no burst" is a real observation rather than a missed one.

What has not been separated is the burst machinery from the external input — the manual-trigger control that would distinguish them (`TRSR,MAN` + `MTRIG`) returned no status, so it is unresolved rather than answered. The leading suspect is **pulse width**: the generator's minimum trigger width is nowhere specified, and 5.7 µs is the narrowest thing this bench can present. Widening it to ~100 µs is the next test, and it needs a one-shot on the credit line rather than any change to the app, since `trigger.extout.pulsewidth` cannot be set.

So the README's standing claim is unchanged and still correct: flow control is **verified electrically but never closed as a loop against a device that waits for it.** What is new is that the reason is now located in the generator's trigger input rather than unknown.

Arming from silence: a blind rate, and a fallback that refuses

Pressing **Capture** before the device has started transmitting arms the analog comparator and waits, rather than recording silence. Two fields on the options screen, both clamped in `serial_core.tsp` as well as on the form, so a programmatic set and a typed one cannot disagree about what is acceptable:

`Options ▸ Arm At`	**0.33 – 6 V**, default **1 V** (`sdec.armlevel`) — how far from the idle level the comparator arms
`Options ▸ Arm Wait`	**2 – 120 s**, default **10 s** (`sdec.armwait`) — and the panel is frozen for every second of it

The 2 s floor is why no pre-existing timing moves. `acq_triggered()` takes this value only where it *exceeds* the mode's own wait, which is `sdec.trigwait` = 3 s for an `edge` capture, so a form wound all the way down is a no-op rather than a way to make captures fail faster than they used to.

The rate is picked blind, and what it buys is window

A line that has not started cannot be measured, so this is the one path that chooses a sample rate before it has any evidence: `sdec.arm_baud` = 38 400 at `sdec.arm_sabit` = 5, which `pick_fs` resolves to **200 kS/s**. The arm fires on the first edge whatever the rate, so **every row below catches the START equally** — what differs is how much of the opening message comes with it. 8N1, at `sdec.n` = 20 000 samples, bytes from `window_bytes()` itself:

`fs`	window	9600	19200	38400	57600
160 kS/s	125 ms	16.7 / 120	8.3 / 240	4.2 / 480	2.8 — refused
200 kS/s	**100 ms**	**20.8 / 96**	10.4 / 192	5.2 / 384	3.5 — refused
320 kS/s	62.5 ms	33.3 / 60	16.7 / 120	8.3 / 240	5.6 / 360

`fs`	window	9600	19200	38400	57600
500 kS/s	40 ms	52.1 / 38	26.0 / 76	13.0 / 153	8.7 / 230

Samples per bit / bytes of window. The two refusals sit below `sdec.minsabit` = 4 and decode at no rate at all.

So the trade is a window length and not a rate ceiling. A hundred milliseconds holds **96 bytes** of a 9600 Bd power-on banner against **38** at 500 kS/s, and what is given up for them is one sample per bit of margin at the fastest rate the row still commits to. A faster line is not abandoned, it loses only its opening bytes: `capture()`'s back-off runs `autoset()` a second time, and by then the device is transmitting, so the ordinary two-pass probe measures it properly. An operator who knows the rate should lock it — a locked rate skips this entirely and captures a full window at that rate's own `fs`.

When no bit time can be measured, uniqueness is what makes a guess safe

If the captured samples yield no bit time, `arm_fit_prior()` re-fits the standard rates in `sdec.arm_try` = {9600, 19200, 28800, 31250, 38400} to the samples **already in hand** — no second capture, so no byte of the opening message is traded for the attempt. Measured offline against the real modules, 1800 bytes of text at 200 kS/s, every candidate forced against every truth, with the pulse-width filter in force. The four rules the measurement separates, over the same 30 pairs:

rule	right	refused	wrong
first candidate that fits, gate 0.25	1	0	**5**
first candidate that fits, gate 0.02	3	1	**2**
uniqueness, gate 0.25	1	4	**1**
uniqueness, gate 0.02 — `sdec.arm_badfrac`, shipped	**3**	**3**	**0**

Per truth, for the two rules that bracket the result:

true rate	first fit at the slack 0.25 gate	uniqueness at 0.02
9600	9600	**9600**
19200	9600 — wrong	**19200**
28800	19200 — wrong	**28800**
31250	28800 — wrong	refused, 2 candidates fit
38400	19200 — wrong	refused, 3 candidates fit
57600	38400 — wrong	refused, nothing fits

`arm_fit_prior()` sets no note when it refuses, and taken alone that reads as a silent failure. It is not one, because the note the arm already wrote is still standing. Measured on the instrument, a real 57 600 Bd device switched on three seconds into the wait leaves the operator with both halves of the answer: the note row reads `armed at 200 kS/s on a line that has not started -- lock the baud rate for a longer window`, and the error row reads `3.5 samples/bit -- 57611 baud needs a faster capture than 200 kSa/s`. The rate is named to within 11 Bd, no rate is published, and `force_baud` is still `nil` — `hw-arm` case N.

Taking the first candidate that framed clean bytes, under the slack gate, was wrong for five of the six truths, and the two worked examples show why that is the normal outcome rather than an edge case. A 19200 Bd line read at 9600 frames **77 bytes with six errors** — a bad fraction of **0.082**, comfortably inside that gate — and the bytes are garbage. The mechanism is structural: half the true rate reads two real frames as one, and the real stop bit lands exactly where the hypothesis looks for its own, so the one check that would notice agrees. A 31250 Bd line read at 28800 frames **258 bytes with zero errors**.

A bad-fraction gate cannot separate the two populations. The true rate gives a bad fraction of exactly **0.000** for all five rates in the ladder. But so do three wrong fits:

wrong candidate that survives the slack 0.25 gate	frames	bad	bad fraction
19200 Bd read at 9600	77	6	0.082
38400 Bd read at 19200	162	36	0.209
57600 Bd read at 38400	351	85	0.239
28800 Bd read at 19200	174	43	0.247
31250 Bd read at 28800	258	0	**0.000**
38400 Bd read at 28800	318	0	**0.000**
38400 Bd read at 31250	318	0	**0.000**

The last three are indistinguishable from the truth by this measure at any threshold — at 5.2 samples per bit three adjacent rates frame 318 of 318 frames with no error each, and only the bytes differ. So **the threshold is what buys the correct answers and uniqueness is what prevents the wrong ones**, and neither substitutes for the other: tightening 0.25 to 0.02 turns two refusals into two right answers, and uniqueness at the slack gate still published 38 400 for a 57 600 Bd line. `sdec.arm_badfrac` = 0.02 against the 0.25 of `relock_badfrac` is the same number answering an opposite question — one asks whether an operator's typed rate has been discredited, where a false alarm discards a deliberate setting; this one asks whether the app may invent a rate nobody asked for, where a false pass is a panel full of confident rubbish.

Before any decode is spent, the detector's own pulse-width battery (`ua_plausible`, `ua_submultiple`, `ua_minrun_absurd`) rejects **10 of the 30 pairs** outright, including 9600 against every truth from 28 800 up.

A refusal is not silence. The two ambiguous cases name the pair, which is what lets the operator resolve it from the one thing the instrument does not know:

	`sdec.probe_note`
published	`no bit time could be measured -- decoded at an ASSUMED 19200 Bd; lock the rate to be sure`
2 or more fit	`this capture fits 28800 Bd AND 31250 Bd equally -- lock the rate in Options to decide`
nothing fits	no note of its own — the capture fails with the decoder's own `no bit time fits the pulse widths`

The pre-trigger count is a percentage of capacity, and `dmm.digitize.count` has no part in it

The template's post-trigger count is **(100 – `position`)% of the buffer's CAPACITY**. Measured over 15 combinations of capacity and position, each matching that rule exactly, with `dmm.digitize.count` set to 77 against a capacity of 1000 changing nothing — the table is in TRIGGER.md §8 and is not repeated here. At `sdec.n` = 20 000 and `sdec.pretrig` = 5:

	readings
the window asked for, `sdec.n`	20 000
capacity requested, `acq_cap()` — the reserve on top of the depth	**21 100**
post-trigger count the template programs, 95 % of capacity	**20 045**
delivered by the instrument	**20 066** — the 20 045 plus 21 pre-trigger samples
the rule the vendor manual implies, `count − count × position/100`	19 000 — short by **5.3 %** of the window

`sdec.n_deliv()` is a **floor, never a ceiling**, which is what makes it safe to answer "was this message seen whole?": the pre-trigger samples land on top of it and are unknowable in advance — a handful on a busy line, up to the full reserve on one that was silent. Free run reserves nothing and delivers `sdec.n` unchanged.

The program shape, and why a shorter one would be worse

`SimpleLoop`	a counted loop with **no wait block at all**, so a recording built on it cannot be armed
`LoopUntilEvent`	buffer clear, an **infinite** digitize, the wait, then the counted burst — what an 8 kB or 32 kB recording loads instead when `sdec.arm_silent()` holds
an `EVENT_ANALOGTRIGGER` wait as **block 1**	`STATE_WAITING` with **zero readings** at 3 s, on a live 1 kHz square wave with the comparator armed dead centre of its 0–3 V swing — against the same signal firing in **under 0.01 s** with a `COUNT_INFINITE` block ahead of the wait

The comparator is derived from the digitizer's sample stream, so with nothing acquiring it never evaluates, and there is no diagnostic — the model simply sits. **The trap is that the broken program looks better:** two blocks instead of four, and on the firmware's low-latency fast path, which it is on because it never does anything. Both results are in TRIGGER.md §7 and §12, with the templates disassembled in §8.

A wait of seconds laps the buffer, and the record comes back rotated

The pre-trigger block writes into the capture buffer for the whole wait, and that buffer is `fillmode = 1` — FILL_CONTINUOUS, which is what keeps event 4915 off the panel. So an arm that waits **laps the ring**, and `readings[1]` is then the ring's physical slot 1 rather than its logical oldest. Measured on an armed capture whose line started 3.1 s after **Capture**, at 200 kS/s. The pre-roll below is shorter than that wait because the digitizer does not start at the key: `autoset()` runs its probe ladder first and arms only once the ladder finds no baud rate, so 2.6 s is measured from the **arm** and 3.1 s from the **press**.

`buf.n` / `buf.capacity`	21 100 / 21 100 — the record is the whole buffer
`buf.startindex` / `buf.endindex`	**13 288** / 13 287
the 1 055-reading pre-trigger silence	at index **13 288**, not at index 1
what the panel showed	`World!Hello, World!…` — **mid-message**, with one framing error where the ring seam fell
2.6 s of pre-roll at 200 kS/s	520 000 readings, or 24 whole laps of the ring plus 13 600

Every byte was present and the order was wrong. The arithmetic is the confirmation: 13 600 predicted against 13 288 measured, which is the same number to within the wait's own jitter.

`buf.startindex` answers it in one proxy read, which is what `sdec.acq_first()` does, and the correction is applied in the three places that read the buffer — the frame copy, the chunked recording reader, and the rate measurement. The last of those mattered as much as the ordering: taken from the ring's physical ends a rotated record spans **minus one sample period**, so `acq_measure_fs()` returned nil and every bit time fell back to the *requested* rate — the one number it exists not to trust. Mutating the correction out gives `acq_fs` = 40 000 against a true 200 000.

Only a full buffer can have wrapped, so the correction is gated on `buf.n` having reached capacity: an armed capture of a busy line fires within about fifteen samples of `initiate()` and comes back `startindex = 1`, and the 2 000-sample level probe that runs into the same buffer afterwards is shorter than its capacity whatever the attribute says.

`buf.n` is not a progress counter on a wrapped buffer, which cost two more defects in the recording poll loop before this was understood: it is pinned at capacity, so "the recording is complete" fired on the same poll the trigger did, and "new samples have arrived" was false for ever after, which ran the idle watchdog on a device that was transmitting. `buf.endindex` keeps moving and is what the loop reads now. At 1 MS/s an 8 kB recording of a 115 200 Bd line fills its buffer in **0.75 s** of wait, so this is the ordinary case at speed rather than a corner.

Endurance: what the soaks measured

The longest run: 15.5 hours unattended, on the instrument itself

10 671 cells over 8 complete laps and 15.5 h, driven entirely by the DMM6500 — the generator selected over the LAN by the instrument, the record written to its own USB key, no host connected after the start:

events instigated	**0** across 10 671 cells
cells with no result	723 — **721 of them refusals the plan's own class allows**
unexpected	**2**: one generator switch, one refusal at the sample-rate ceiling
confidently wrong bytes	**0**
holds, unrecorded cells	0, 0
late waveform switches recovered	**18 of 19**

Zero events is the one that matters, because on this firmware an event is a modal box on the panel and nothing suppresses it — `localnode.showevents` governs the remote interface only. A run nobody is watching must instigate none, and 10 671 cells instigated none.

The two unexpected cells are different things and the record separates them. One was the generator failing to change waveform after three attempts, counted as `SDG failed` rather than as a decode failure. One was the app declining to decode, which at 4–6.5 samples a bit is a reasonable answer — see **The rate ceiling** in BENCH.md. Neither was a wrong byte.

18 of 19 late switches recovered is the ARWV retry measured in use rather than inferred from an absence: `brun.nselback` counts cells whose waveform needed a second selection. A lap makes ~233 real switches and about 10 % of them land late, so ~24 were expected; 19 occurred and 18 were repaired. An unrepaired

one costs the cell, not the run.

The host-driven soaks

Three 17-hour soaks, each **6 laps of the full 1 683-point matrix, 10 098 capture-and-decode cycles on one power cycle** — 318, 313 and 301 failures, listed in the README. Everything below is the shape of the **two whose per-cell records are retained**, which agree within **0.2 % on duration and 1.6 % on failure count**.

> **Read 318 and 313 as upper bounds.** A LIN frame opens with a break — a dominant interval longer than any UART character — which the decoder flags, correctly, while the oracle holds the byte the wire carried. A byte-exact LIN capture can therefore fail on flag count alone, and in these two runs **493 of 494 and 363 of 364 flag-budget failures are LIN**, ~120 a lap over v61, v62 and v63. The judge substitutes the break byte where the oracle says one belongs; these two runs' logs are not retained parseably, so their counts are not recomputed. Read `v63`'s place in the concentration below the same way.

Lap time is the leak indicator, because a leaked display object, an unreturned buffer handle or a fragmenting allocator wedges the instrument around hour six without failing a single point. It fell in both runs. The second's six laps ran **10 567, 10 219, 10 254, 10 282, 10 173 and 10 299 s**, each within ±1 % of its counterpart in the first.

3.1 % of points fail, against a matrix built to attack the decoder — LIN traffic presented to a UART decoder, single-byte and walking-bit patterns, sinusoidal drift, and swings and offsets swept to the edge of range. **That figure blends two unrelated failures**, and the split is under **Rate detection and decode, counted separately** above: on the current build almost all of it is the *reported rate* being wrong, not a byte. These runs' logs cannot be re-split, so the blended figure is all they support. It is not comparable to the rate-detection figures under **Known failures of automatic rate detection** in the manual. Two properties:

- **It concentrates.** Four vectors — `v94`, `v63`, `v90`, `v71` — account for **76 %** of all failures; `v63` is a LIN frame whose 13-bit break field is not valid 8N1.
- **It is mostly intermittent.** The 318 failures are **162 distinct cells**, of which **88 failed in exactly one lap of the six** and **6 in all six**.

Cross-run agreement is stronger than within-run agreement. Comparing which fixed-stimulus cells failed, a lap of the second run differs from its counterpart in the first by **11–16 cells**, against **21–32** for every pair of laps inside the first run. The failure set is a property of the matrix, not of the run — which is what licenses reading a cell that fails in all six laps as a property of the cell.

A third, 7-hour soak covers the **recording** path, which neither long soak exercises: **81 laps, 3 726 cycles and 8 one-press recordings, zero failures**, every recording ending on its byte ceiling.

Not covered by any soak: many power cycles rather than one, and front-panel interaction — that is `hw-panel`'s job, under **The release gate, stage by stage** in BENCH.md.

Firmware limits

One app build per power cycle. `sdec.start()` builds the display objects and the firmware does not appear to return the pool fully, so after enough rebuilds in one power cycle the instrument reports "*the maximum number of objects have already been created*" and the app refuses to build rather than coming up

with controls missing. How many is enough is **not characterised** — it has been seen after a few dozen. One relaunch is safe; a long editing session needs a power cycle. This affects development, not normal use.

The build itself is **122 display objects** live after `ui_build()`, and **134** once the options screen has been built too. Counted with the harness census, not estimated.

The panel geometry is baked in rather than negotiated at runtime, and it does not need to be: the panel is pixel-identical across the entire TSP range. Object `y` is relative to a content area **49 px below the panel top**, and a screen title is limited to **31 characters**. `serial_ui.tsp` is 1 300 lines built against those constants, and they carry to another TSP model unmodified — so a port is an acquisition question only, never a layout one.

The display-side unknown is not geometry but the **object pool** above: an allocation limit, not a dimension, and uncharacterised on every model including this one.

Event 4915 — "attempting to store past the capacity of a reading buffer" — is ERROR severity, which the front panel shows as a **modal dialog over the app** whatever `localnode.showevents` says. Two defences apply to every armed capture: `fillmode = 1` per capture (the numeric value; `buffer.FILL_CONTINUOUS` does not exist on this firmware and assigning it raises), and 100 readings of capacity past the count the trigger model is asked for. Measured with no headroom: 20–30 events per capture.

`localnode.showevents` governs the remote interface only. It cannot suppress the panel's dialog, so muting is not a defence — not posting the event is.